

Fundamentals of Object-Oriented Programming

Procedure-Oriented Programming Vs. Object-Oriented Programming (OOP)

1. Introduction :

Programming languages दो मुख्य paradigms पर आधारित होती हैं - Procedure-Oriented Programming (POP) और Object-Oriented Programming (OOP). POP में program को functions (procedures) के रूप में लिखा जाता है जबकि OOP में program को objects और classes के रूप में design किया जाता है। OOP, POP का advanced रूप है जो real-world entities को better तरीके से model करने के लिए उपयोग किया जाता है।

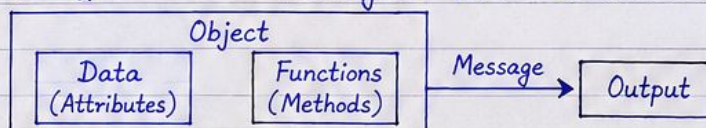
2. Procedure-Oriented Programming (POP) :

इस approach में program को छोटे-छोटे functions (procedures) में divide किया जाता है। Data और functions अलग-अलग होते हैं। Data पर functions operate करते हैं।



3. Object-Oriented Programming (OOP) :

इस approach में program को objects और classes के रूप में design किया जाता है। यहाँ Data और functions एक साथ (encapsulated) होते हैं। Objects real-world entities को represent करते हैं और एक-दूसरे के साथ messages के द्वारा communicate करते हैं।



4. POP vs OOP - तुलना (Comparison)

S.No.	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
1.	Program को functions (procedures) में divide किया जाता है।	Program को objects और classes के रूप में design किया जाता है।
2.	Data और functions अलग-अलग होते हैं।	Data और functions एक साथ (encapsulated) होते हैं।
3.	Data पर functions operate करते हैं।	Functions, data के साथ operate करते हैं।
4.	Data security कम होती है।	Data security अधिक होती है (Encapsulation के कारण)।
5.	Program structure कम organized होता है।	Program structure अधिक organized और modular होता है।
6.	Real-world entities को model करना कठिन होता है।	Real-world entities को model करना आसान होता है।
7.	Code reuse नहीं के बराबर होता है।	Code reuse (Inheritance, Polymorphism) के द्वारा संभव है।
8.	Maintenance कठिन और समय लेने वाला होता है।	Maintenance आसान और समय बचाने वाला होता है।

5. OOP की मुख्य विशेषताएँ (Features) :

- Class & Object
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing
- Modularity
- Reusability

CHAPTER - OBJECT ORIENTED PROGRAMMING CONCEPTS

1. Classes :

- Class एक user-defined blueprint या template होता है, जिससे objects बनाए जाते हैं।
- Class में data members (variables) और member functions (methods) को define किया जाता है।
- यह केवल logical entity होती है, memory allocate नहीं होती।
- Class के द्वारा similar type के objects का group बनाया जाता है।
- Class में access specifiers का use करके data को protect किया जाता है।

Class : Student
- int rollNo;
- string name;
- string course;
+ void input();
+ void display();

Syntax :

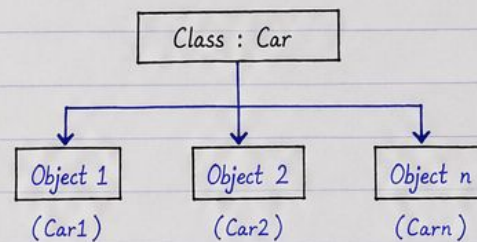
```
class ClassName {  
    // data members  
    data_type variable1;  
    data_type variable2;  
    // member functions  
    return_type function1();  
    return_type function2();  
};
```

} → Data Members
(Variables)

} → Member Functions
(Methods)

2. Object :

- Object, class का एक instance होता है।
- जब class को memory में allocate किया जाता है, तब object बनता है।
- Object real-world entity को represent करता है और class में define properties और behaviour को use करता है।
- हर object की अपनी identity, state और behaviour होती है।



Syntax :

```
ClassName objectName = new ClassName();
```

Example :

```
Car c1 = new Car();  
Car c2 = new Car();
```

3. Object Reference :

- Object reference एक variable होता है जो object के address को store करता है।
- इसी reference के द्वारा हम object के members (data और functions) को access करते हैं।
- Reference का उपयोग करके एक object से दूसरे object में value copy की जा सकती है।

Reference

carRef
1001

Memory

Address : 1001
Car Object
regNo : UP78AB1234
model : Swift
color : White
price : 600000

Syntax :

```
ClassName referenceName = objectName;
```

Example :

```
Car car1 = new Car();  
Car carRef = car1; // carRef now refers to car1
```

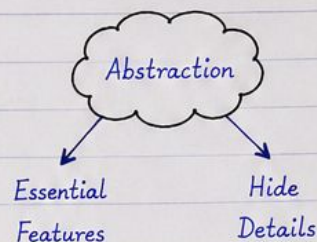
Note :

Object reference को pointer के तरह काम करता है, लेकिन OOP में इसे reference कहा जाता है।

CHAPTER - OBJECT ORIENTED PROGRAMMING CONCEPTS

4. Abstraction :

- Abstraction का मतलब है केवल जरूरी information को दिखाना और unnecessary details को hide करना।
- यह complexity को reduce करता है और system को easy to use बनाता है।
- Java में abstraction को abstract class और interface के द्वारा achieve किया जाता है।



Syntax (Abstract Class) :

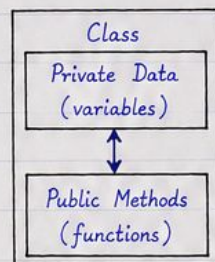
```
abstract class ClassName {  
    abstract returnType methodName(parameters);  
    // concrete methods  
}
```

Example :

```
abstract class Shape {  
    abstract void draw(); // only declaration  
}
```

5. Encapsulation :

- Encapsulation का मतलब है data (variables) और methods को एक unit (class) में बाँधना।
- यह data को unauthorized access से protect करता है।
- Private members के द्वारा data hiding की support करता है।
- Public methods के द्वारा controlled access दिया जाता है।



Syntax :

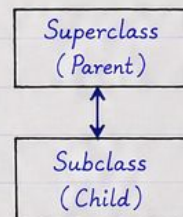
```
class ClassName {  
    private data members;  
    public void setData(...) { ... }  
    public dataType getData() { ... }  
}
```

Example :

```
class Student {  
    private int rollNo;  
    private String name;  
    public void setName(String n) { name = n; }  
    public String getName() { return name; }  
}
```

6. Inheritance :

- Inheritance एक mechanism है जिसमें एक class (child/subclass) दूसरी class (parent/superclass) के properties और methods को inherit करती है।
- इससे code reusability बढ़ती है।
- Java में "extends" keyword का उपयोग inheritance के लिए होता है।



Syntax :

```
class SubclassName extends SuperclassName {  
    // additional members  
}
```

Example :

```
class Animal {  
    void eat() { System.out.println("Eating..."); }  
}  
class Dog extends Animal {  
    void bark() { System.out.println("Barking..."); }  
}
```

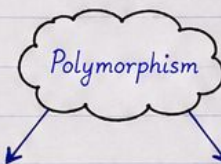
Note :

- Abstraction अनावश्यक details को hide करता है।
- Encapsulation data को secure करता है।
- Inheritance code reuse के लिए helpful है और relationship को represent करता है।

CHAPTER - OBJECT ORIENTED PROGRAMMING CONCEPTS

7. Polymorphism :

- Polymorphism का मतलब है "one name, many forms".
- इसमें एक ही method अलग-अलग objects के लिए अलग behavior दिखा सकता है।
- Java में polymorphism दो प्रकार का होता है -
 - (i) Compile-time Polymorphism (Method Overloading)
 - (ii) Run-time Polymorphism (Method Overriding)



(i) Compile-time Polymorphism (Method Overloading):

- यह उसी class में multiple methods बनाने से होता है जिनका नाम same होता है लेकिन parameter list अलग होती है।
- Compiler यह decide करता है कि किस method को call करना है।

Example :

```
class Calculator {  
    int add(int a, int b) { ... }  
    int add(int a, int b, int c) { ... }  
    double add(double a, double b) { ... }  
}
```

(ii) Run-time Polymorphism (Method Overriding):

- यह inheritance के द्वारा achieve होता है।
- Parent class का method child class में same signature के साथ फिर से define किया जाता है।
- JVM run-time पर decide करता है कि किस class का method execute करना है (dynamic binding)।

Example :

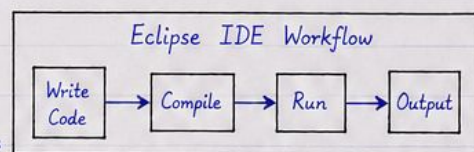
```
class Animal {  
    void sound() { ... }  
}  
class Dog extends Animal {  
    void sound() { ... } // overriding  
}
```

8. Introduction of Eclipse (IDE) for Developing Programs in Java :

- Eclipse एक popular IDE (Integrated Development Environment) है जो Java programs को develop, compile, run और debug करने के लिए उपयोग किया जाता है।

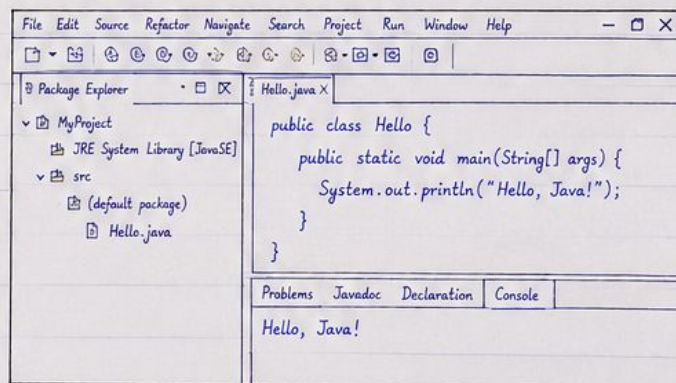
Eclipse की विशेषताएँ (Features):

- Easy project creation
- Powerful code editor
- Syntax highlighting
- Compile-time error checking
- Debugging tools (Breakpoints, Step Into, Step Over)
- Supports plugins
- Integrated JRE
- Code completion & suggestions
- Smart refactoring
- Easy GUI for development



Eclipse में Java Program बनाने के Steps :

1. Eclipse IDE को download और install करें।
2. Workspace select करें (जहाँ projects store होंगे)।
3. File → New → Java Project पर click करें।
4. Project name दें और Finish करें।
5. src folder के अंदर package बनाएँ (यदि required हो)।
6. New → Class बनाकर Java code लिखें।
7. Program को Run (▶) करें और Output console में देखें।



Note : Eclipse Java development को आसान बनाता है और बड़े projects को manage करने में बहुत helpful होता है।